

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Нейронные сети»
Тема: Регрессионная модель изменения цен на дома в Бостоне

Студент гр. 1306

Шаврин А.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Реализовать предсказание медианной цены на дома в пригороде Бостона в середине 1970-х по таким данным, как уровень преступности, ставка местного имущественного налога и т.д. Данный набор содержит относительно немного образцов данных: всего 506, разбитых на 404 обучающих и 102 контрольных образца. И каждый признак во входных данных (например, уровень преступности) имеет свой масштаб. Например, некоторые признаки являются пропорциями и имеют значения между 0 и 1, другие — между 1 и 12 и т. д. Не путайте регрессию с алгоритмом логистической регрессии. Как ни странно, логистическая регрессия не является регрессионным алгоритмом — это алгоритм классификации.

Задание.

- Ознакомиться с задачей регрессии
- Изучить отличие задачи регрессии от задачи классификации
- Создать модель
- Настроить параметры обучения
- Обучить и оценить модели
- Ознакомиться с перекрестной проверкой

Выполнение работы.

Данные загружаются с помощью встроенной функции `boston_housing.load_data()` из Keras. Обучающая выборка содержит 404 образца, тестовая — 102. Каждый образец представлен 13 признаками.

Для нормализации данных используется стандартизация (вычитание среднего и деление на стандартное отклонение), рассчитанное по обучающей выборке и примененное к обеим выборкам (рисунок 1).

```

  ▾ Загрузка данных

[5] ✓ 0s
(train_data, train_targets), (test_data, test_targets) = boston_housing.load_data()

print("Размер обучающей выборки:", train_data.shape)
print("Размер тестовой выборки:", test_data.shape)
print("Первые 5 целевых значений (тыс. $):", test_targets[:5])

  ▾
    Размер обучающей выборки: (404, 13)
    Размер тестовой выборки: (102, 13)
    Первые 5 целевых значений (тыс. $): [ 7.2 18.8 19.  27. 22.2]

  ▾ Предобработка данных

[7] ✓ 0s
mean = train_data.mean(axis=0)
train_data -= mean
std = train_data.std(axis=0)
train_data /= std

test_data -= mean
test_data /= std

```

Рисунок 1. Чтение и предобработка данных.

Для проведения экспериментов была написана функция `build_model()` (рисунок 2). Модель представляет собой полносвязную нейронную сеть с двумя скрытыми слоями по 64 нейрона с функцией активации ReLU. Выбор архитектуры обусловлен необходимостью моделировать нелинейные зависимости в данных Boston Housing, содержащем 13 признаков.

Особенности задачи регрессии

В отличие от задач классификации, где целевая переменная дискретна (метки классов), в регрессии требуется предсказать непрерывное числовое значение. Это определяет два ключевых аспекта модели:

- Выходной слой: используется линейная активация (без функции активации), поскольку предсказание может принимать любые вещественные значения. В задачах классификации выходной слой обычно применяет softmax (для многоклассовой) или sigmoid (для бинарной), чтобы получить вероятности принадлежности классам.
- Функция потерь: выбрана среднеквадратичная ошибка (MSE), которая штрафует большие ошибки квадратично, что способствует более быстрой сходимости и чувствительности к выбросам. Для классификации стандартом является кросс-энтропия.

В качестве метрики качества используется средняя абсолютная ошибка (MAE), поскольку она интерпретируема в исходных единицах целевой переменной (тыс. долларов) и менее чувствительна к выбросам, чем MSE.

Оптимизатор - RMSprop, который хорошо зарекомендовал себя для задач регрессии благодаря адаптивной скорости обучения.



Построение модели

```
[8] ✓ Os
def build_model():
    model = Sequential()
    model.add(Dense(64, activation='relu', input_shape=(train_data.shape[1],)))
    model.add(Dense(64, activation='relu'))
    model.add(Dense(1))
    model.compile(optimizer=RMSprop(), loss='mse', metrics=['mae'])
    return model
```

Рисунок 2. Функция создания модели.

Функция `kfold_cross_validation` реализует K-fold разбиение обучающей выборки, обучение модели на каждом фолде и сохранение истории обучения. Это позволяет в дальнейшем анализировать динамику метрик (рисунок 3).

Функция K-fold кросс-валидации

```
[17]
✓ Os
def kfold_cross_validation(K, epochs, verbose=0):
    """
    Выполняет K-fold кросс-валидацию и возвращает список объектов history для каждого фолда.
    """
    num_val_samples = len(train_data) // K
    histories = []

    for i in range(K):
        print(f'Фолд №{i+1}/{K}')
        # Валидационный блок
        val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
        val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]

        # Обучающие блоки (все остальные)
        partial_train_data = np.concatenate(
            [train_data[:i * num_val_samples],
             train_data[(i + 1) * num_val_samples:]],
            axis=0
        )
        partial_train_targets = np.concatenate(
            [train_targets[:i * num_val_samples],
             train_targets[(i + 1) * num_val_samples:]],
            axis=0
        )

        model = build_model()
        history = model.fit(
            partial_train_data, partial_train_targets,
            validation_data=(val_data, val_targets),
            epochs=epochs, batch_size=1, verbose=verbose
        )
        histories.append(history)

    return histories
```

Рисунок 3. Функция K-fold обучения.

1. Определение оптимального числа эпох

Для $K=4$ была проведена кросс-валидация на 100 эпохах. На рис. 4 представлен график средней абсолютной ошибки на валидации (val_mae) по эпохам для каждого фолда и усредненная кривая. На рис. 5 показаны потери на валидации (val_loss) с пропуском первых 10 эпох для лучшей визуализации. Для более точного определения оптимального числа эпох была построена сглаженная кривая усредненной MAE (окно сглаживания 15, пропущены первые 10 эпох). Минимум достигается на 71-й эпохе (рис. 6).

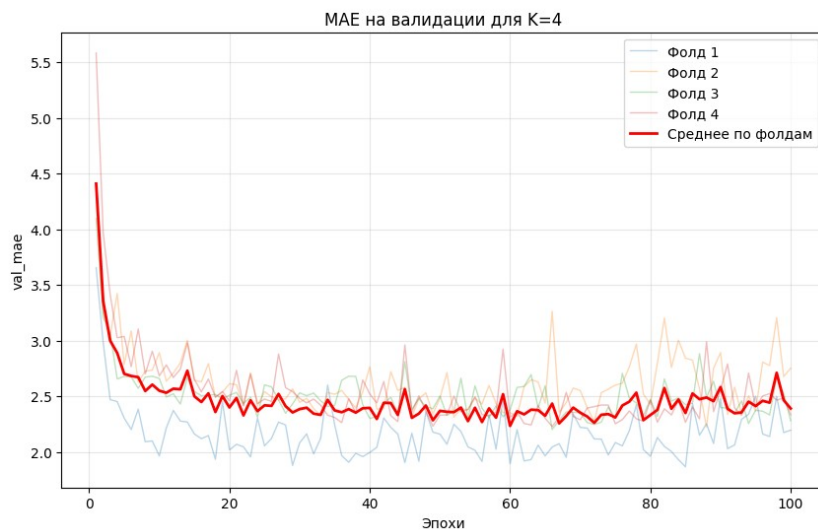


Рисунок 4. График средней абсолютной ошибки

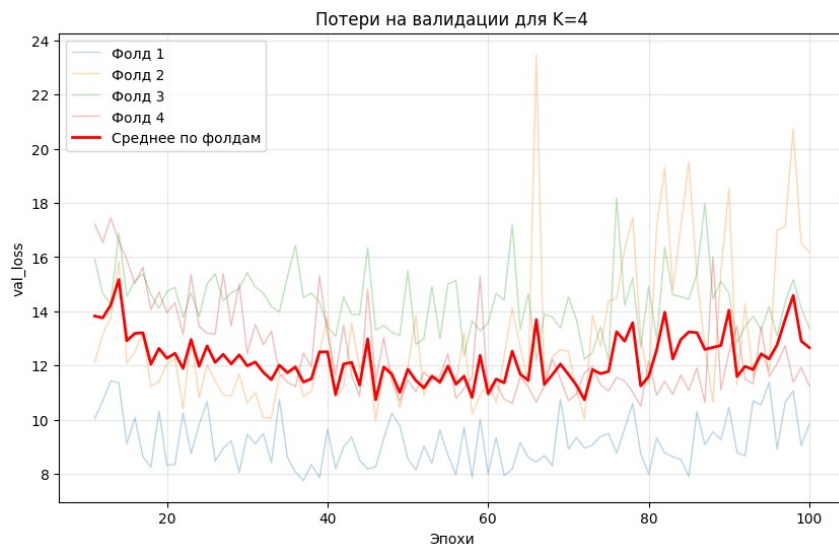


Рисунок 5. График потери



Рисунок 6. График сглаженной кривой усредненной MAE.

Таким образом, оптимальное число эпох для данной модели и параметра $K=4$ составляет 71.

2. Исследование влияния параметра K

Для фиксированного числа эпох (71) была проведена K -fold кросс-валидация с различными значениями K : 2, 3, 4, 5, 6. Для каждого K вычислялась средняя MAE на последней эпохе по всем фолдам. Результаты приведены на рисунке 7. На рис. 8 - 11 представлены графики MAE и потерь на валидации для каждого K при 71 эпохе.



Рисунок 7. График зависимости усредненной MAE от числа блоков K .

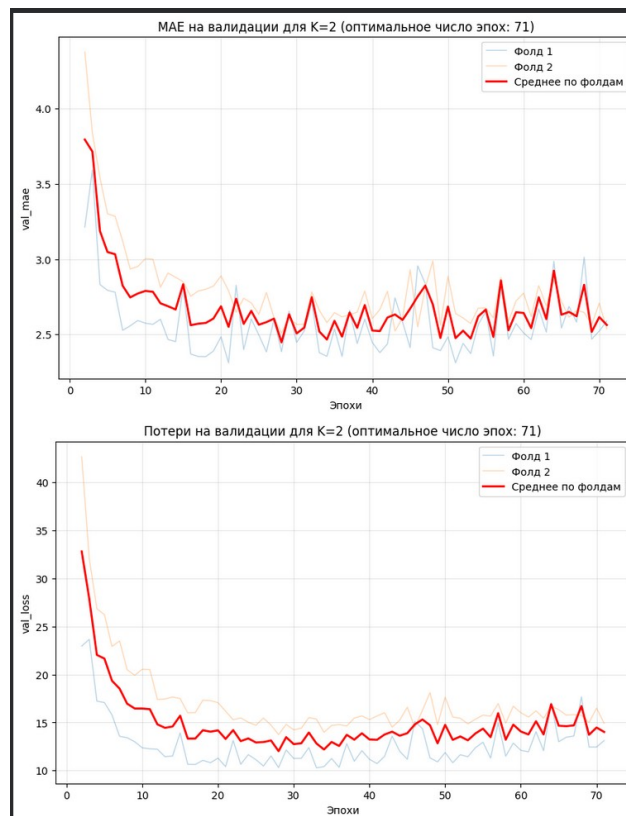


Рисунок 7. График зависимости усредненной MAE и потерь для 2х блоков.

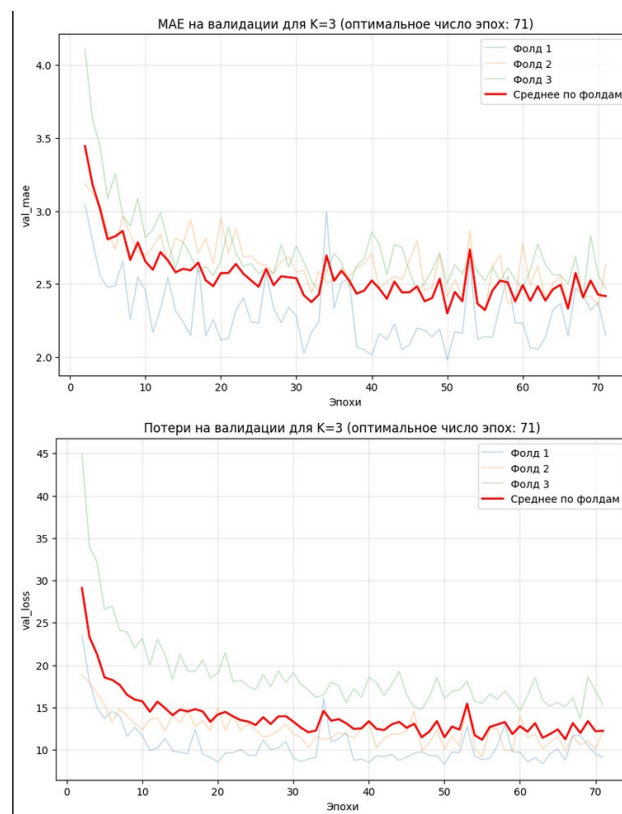


Рисунок 8. График зависимости усредненной МАЕ и потерь для 3х блоков.

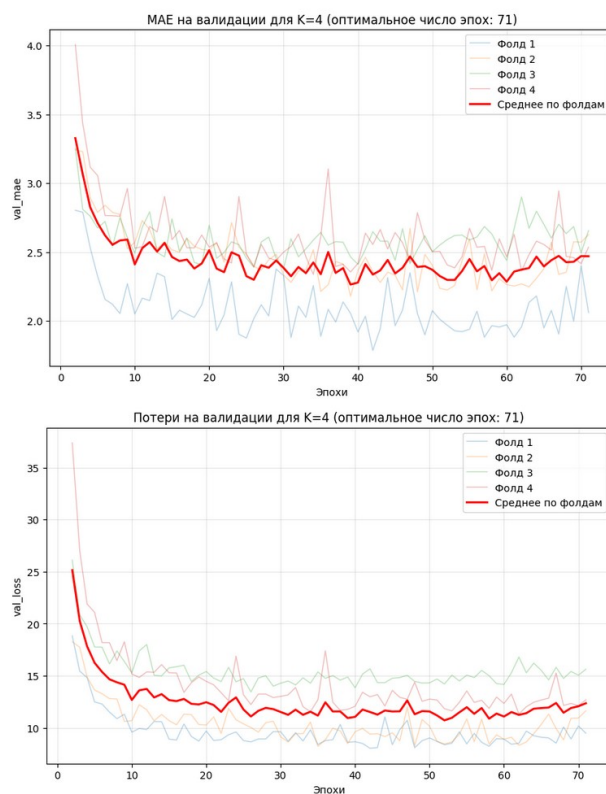


Рисунок 9. График зависимости усредненной МАЕ и потерь для 4х блоков.

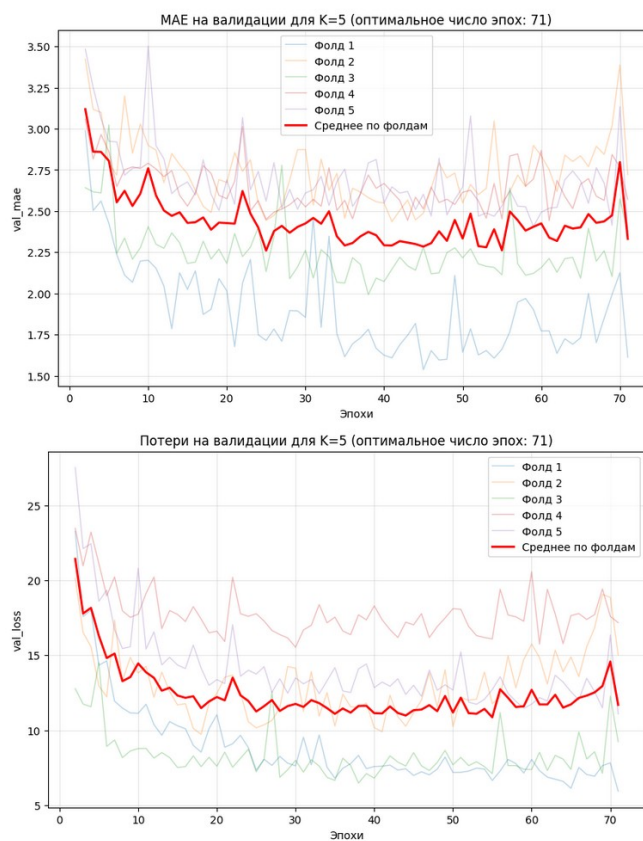


Рисунок 10. График зависимости усредненной MAE и потерь для 5и блоков.

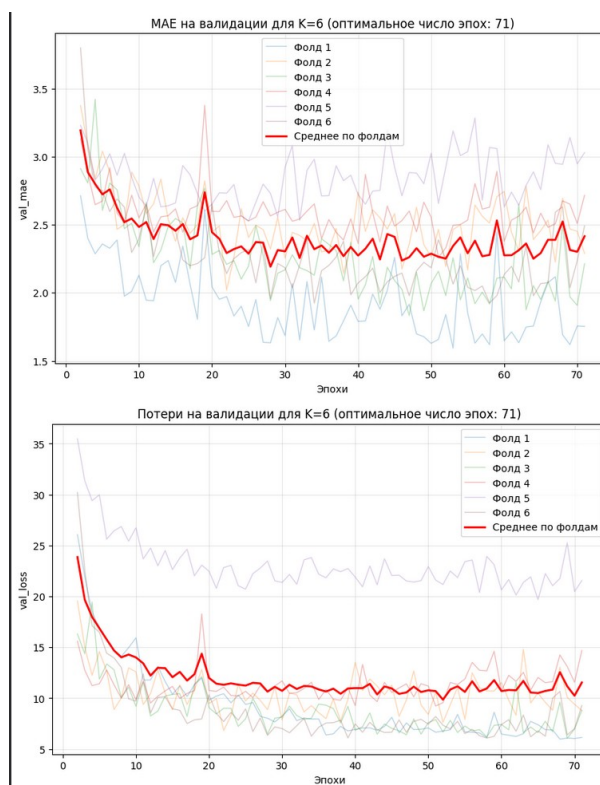


Рисунок 11. График зависимости усредненной MAE и потерь для 6и блоков.

Анализ графиков показывает, что наименьшая средняя ошибка достигается при $K=5$ ($MAE = 2.33$). При увеличении K до 6 ошибка незначительно возрастает. Это может объясняться тем, что при слишком большом числе фолдов ($K=6$) объем обучающей выборки на каждом шаге становится меньше, что приводит к некоторому увеличению дисперсии оценки. При малых K (2–3) оценка менее стабильна, и ошибка выше.

При $K=5$ разброс значений MAE между фолдами был наименьшим, что указывает на более устойчивую оценку качества модели. При $K=2$ разброс максимален, что делает оценку ненадежной.

Выводы.

В ходе выполнения лабораторной работы были получены следующие результаты:

- Разработана полносвязная нейронная сеть с двумя скрытыми слоями по 64 нейрона. Предобработка данных (стандартизация) выполнена корректно: параметры нормализации вычислены только по обучающей выборке и применены к тестовой, что исключает подглядывание в тестовые данные.
- С помощью K-fold кросс-валидации ($K=4$) были построены кривые обучения. Анализ графиков MAE и loss на валидации показал, что после определенного этапа (примерно 30–40 эпох) ошибка перестает существенно снижаться и начинает флуктуировать, что свидетельствует о начале переобучения. Применение сглаживания (окно 15) позволило более точно локализовать минимум усредненной MAE. Оптимальное число эпох составило 71. Это значение обеспечивает наилучшее соотношение качества обучения и обобщающей способности.
- Проведена серия экспериментов с различными значениями K (от 2 до 6) при фиксированном числе эпох (71). $K=2$ дал наибольшую ошибку (2.56), что объясняется малым объемом валидационной выборки и, как следствие, высокой дисперсией оценки. $K=3$ и $K=4$ показали промежуточные результаты (2.42 и 2.47 соответственно). $K=5$ оказался оптимальным (2.33), обеспечивая наилучший компромисс между смещением и дисперсией. $K=6$ привел к незначительному росту ошибки (2.42), что может быть связано с уменьшением обучающей выборки на каждом шаге и увеличением разброса оценок между фолдами.

Таким образом, правильный выбор числа фолдов в кросс-валидации и определение момента остановки обучения являются ключевыми факторами для построения модели с хорошей обобщающей способностью.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab3.ipynb

```
"""# Установка зависимостей
"""
```

```
!pip install pandas numpy matplotlib scikit-learn tensorflow
```

```
"""# Импорт"""
```

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import boston_housing
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import RMSprop
from scipy.signal import savgol_filter
```

```
"""# Загрузка данных"""
```

```
(train_data, train_targets), (test_data, test_targets) = boston_housing.load_data()
```

```
print("Размер обучающей выборки:", train_data.shape)
print("Размер тестовой выборки:", test_data.shape)
print("Первые 5 целевых значений (тыс. $):", test_targets[:5])
```

```
"""# Предобработка данных
```

```
"""
```

```
mean = train_data.mean(axis=0)
train_data -= mean
std = train_data.std(axis=0)
train_data /= std
```

```
test_data -= mean
test_data /= std
```

```
"""# Построение модели
```

```
"""
```

```
def build_model():
    model = Sequential()
    model.add(Dense(64, activation='relu', input_shape=(train_data.shape[1],)))
    model.add(Dense(64, activation='relu'))
    model.add(Dense(1))
    model.compile(optimizer=RMSprop(), loss='mse', metrics=['mae'])
    return model
```

```
"""# Функция K-fold кросс-валидации"""
```

```
def kfold_cross_validation(K, epochs, verbose=0):
```

```
    """
```

Выполняет K-fold кросс-валидацию и возвращает список объектов history для каждого фолда.

```
    """
```

```
    num_val_samples = len(train_data) // K
    histories = []
```

```
    for i in range(K):
```

```
        print(f'Фолд №{i+1}/{K}')
        # Валидационный блок
```

```
        val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
```

```
        val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]
```

```
        # Обучающие блоки (все остальные)
```

```
        partial_train_data = np.concatenate(
            [train_data[:i * num_val_samples],
             train_data[(i + 1) * num_val_samples:]],
            axis=0
```

```
        )
```

```
        partial_train_targets = np.concatenate(
            [train_targets[:i * num_val_samples],
             train_targets[(i + 1) * num_val_samples:]],
            axis=0
```

```
        )
```

```
        model = build_model()
```

```
        history = model.fit(
            partial_train_data, partial_train_targets,
            validation_data=(val_data, val_targets),
            epochs=epochs, batch_size=1, verbose=verbose
        )
```

```
        histories.append(history)
```

```
    return histories
```

```
"""# Определение оптимального числа эпох (точка переобучения)"""
```

```
K = 4
```

```
epochs_to_test = 100
```

```
print(f"K-fold кросс-валидация с K={K} для анализа числа эпох")
```

```
histories = kfold_cross_validation(K, epochs_to_test, verbose=0)
```

```
def plot_kfold_metric(histories, metric_name='val_mae', title='Динамика метрики',
smooth_window=0, start_epoch=0, optimum=False):
```

```
    """
```

Строит графики указанной метрики для каждого фолда и усредненную кривую, начиная с эпохи start_epoch+1 (если start_epoch > 0).

Параметры:

```

- histories: список объектов history от K-fold
- metric_name: название метрики ('loss', 'mae', 'val_loss', 'val_mae')
- title: заголовок графика
- smooth_window: размер окна для сглаживания 0 - не применять сглаживание
- start_epoch: с какой эпохи начинать отображение (0 – все эпохи)
- optimum: показывать ли автоматически определенное оптимальное число эпох
"""
plt.figure(figsize=(10, 6))

curves = [h.history[metric_name] for h in histories]
epochs = len(curves[0])

if start_epoch > 0:
    x_range = range(start_epoch + 1, epochs + 1)
    curves_cut = [c[start_epoch:] for c in curves]
else:
    x_range = range(1, epochs + 1)
    curves_cut = curves

for i, curve in enumerate(curves_cut):
    plt.plot(x_range, curve, alpha=0.3, linewidth=1, label=f'Фолд {i+1}')

avg_curve = np.mean(curves_cut, axis=0)
if smooth_window != 0:
    avg_curve = savgol_filter(avg_curve, window_length=smooth_window, polyorder=2)

plt.plot(x_range, avg_curve, color='red', linewidth=2, label='Среднее по фолдам')

if optimum:
    min_idx = np.argmin(avg_curve)
    optimal_epochs = start_epoch + min_idx + 1

    plt.axvline(x=optimal_epochs, color='green', linestyle='--', linewidth=2,
                label=f'Оптимум: {optimal_epochs} эпох')

plt.xlabel('Эпохи')
plt.ylabel(metric_name)
plt.title(title)
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

return np.mean(curves, axis=0)

avg_val_mae = plot_kfold_metric(histories, metric_name='val_mae',
                               title=f'MAE на валидации для K={K}')

avg_val_loss = plot_kfold_metric(histories, metric_name='val_loss',
                                 title=f'Потери на валидации для K={K}', start_epoch=10)

avg_val_mae_avg = plot_kfold_metric(histories, metric_name='val_mae',
                                    title=f'Сглаженная усредненная MAE (пропущены первые 10 эпох)',

```

```

smooth_window=15, start_epoch=10, optimum=True)

"""# Исследование влияния параметра K в K-fold проверке"""

K_values = [2, 3, 4, 5, 6]
optimal_epochs = 71
results = {}
all_histories_by_K = {}
for k in K_values:
    print(f"Вычисление для K={k}...")
    histories_k = kfold_cross_validation(k, optimal_epochs, verbose=0)
    all_histories_by_K[k] = histories_k
    final_maes = [h.history['val_mae'][-1] for h in histories_k]
    avg_mae = np.mean(final_maes)
    results[k] = avg_mae
    print(f"\tСредняя MAE при K={k}: {avg_mae:.4f}")

for k, histories_k in all_histories_by_K.items():
    plot_kfold_metric(histories_k, metric_name='val_mae',
                      title=f'MAE на валидации для K={k} (оптимальное число эпох:
{optimal_epochs})', start_epoch=1)
    plot_kfold_metric(histories_k, metric_name='val_loss',
                      title=f'Потери на валидации для K={k} (оптимальное число эпох:
{optimal_epochs})', start_epoch=1)

def plot_k_comparison(results, title='Сравнение средней MAE при разных K'):
    """Столбчатая диаграмма для сравнения результатов K-fold при разных K."""
    plt.figure(figsize=(8, 5))
    plt.bar(results.keys(), results.values(), color='skyblue')
    plt.xlabel('K')
    plt.ylabel('Средняя MAE')
    plt.title(title)
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()

plot_k_comparison(results, title='Зависимость средней MAE от числа блоков K')

"""# Финальное обучение на всех обучающих данных и оценка на тесте"""

optimal_epochs = 71
final_model = build_model()
final_model.fit(train_data, train_targets,
                epochs=optimal_epochs, batch_size=1, verbose=0)

test_loss, test_mae = final_model.evaluate(test_data, test_targets, verbose=0)
print(f"Итоговая MAE на тестовой выборке: {test_mae:.4f}")
print(f"Это соответствует средней ошибке {test_mae*1000:.0f}")

```